

Memory Safety without Lifetime Parameters

Document #: P3444R0
Date: 2024-10-15
Project: Programming Language C++
Audience: SG23
Reply-to: Sean Baxter
<seanbax.circle@gmail.com>

§ 1 Abstract

This proposal describes the implementation of a memory-safe reference type that does not use lifetime annotations. The goal of the proposal is to:

1. Explore the viability of memory safety without lifetime parameters.
2. Explain the infeasibility of making legacy lvalue- and rvalue-references memory safe.
3. Address common objections to safety models that use lifetime annotations.

§ 2 Safe references

“Safe C++”[[safecpp](#)] introduced a comprehensive design for compile-time memory safety in C++. The borrow checking model in Safe C++ requires lifetime parameters, a feature that increases expressiveness but complicates the language’s type system. This proposal describes an alternative style of borrow checking, guaranteeing lifetime safety without the involvement of lifetime annotations.

First let’s recap how lifetime parameters are declared and used.

[lifetimes1.cxx](#) – (Compiler Explorer)

```
#feature on safety

// Function parameters have different lifetime parameters.
// Return type is constrained by x's lifetime.
auto f1/(a, b)(int^/a x, int^/b y, bool pred) safe -> int^/a {
    // Error:
    // function auto f1/(a, b)(int^/a, int^/b) -> int^/a returns
    // object with lifetime b, but b doesn't outlive a
    // return y;
```

```

    return pred ? x : y;
}

// Function parameters have a common lifetime parameter.
auto f2(/a)(int^/a x, int^/a y, bool pred) safe -> int^/a {
    // Ok
    return pred ? x : y;
}

// Error:
// cannot use lifetime elision for return type int^
auto f3(int^ x, int^ y) safe -> int^;

```

In Safe C++, occurrences of the borrow type $T^$ in function declarations and in data members require specialization with *lifetime arguments*. Lifetime arguments name *lifetime-parameters* declared as part of the function declaration. Borrow types without lifetime arguments have *unbound lifetimes* and borrows with lifetime arguments have *bound lifetimes*. These are treated as different entities by the language's type system, and there are subtle rules on how bound lifetimes decay to unbound lifetimes and how unbound lifetimes become bound. Lifetime annotations greatly improve the capability of safe references, but extend an already complicated type system.

The above code declares functions `f1`, `f2` and `f3` with *lifetime-parameter-lists*. Borrows in function return types must be constrained by the lifetimes of one or more function parameters. Failure to match lifetime arguments between function parameters and return types will cause a borrow checker failure. `f1` fails to borrow check because the returned parameter `y` does not outlive the lifetime `/a` on the return type.

Elision rules make lifetime annotations implicit in some cases. But elision can fail, requiring users to intervene with annotations. In the example above, the declaration of `f3` fails because the elision rules cannot determine the lifetime argument on the returned borrow.

[lifetimes2.cxx – \(Compiler Explorer\)](#)

```

#feature on safety

// New elision rules:
// All parameters are constrained by a common lifetime.
// The common lifetime constrains the return type.
int% f4(int% x, int% y, bool pred) safe {
    // Can return either x or y, because they outlive the common lifetime
    // and the common lifetime outlives the result object.
    return pred ? x : y;
}

```

This proposal introduces a new *safe reference* marked by the reference declarator $T\%$. Safe references do not take lifetime arguments and there is no notion of *bound* or *unbound* lifetimes. The lifetime parameterization is determined by the formation of the function type. For a free function, all function parameters outlive a single invented lifetime that extends through the duration of the function call. For a non-static member function with

the `%ref-qualifier`, the implicit object parameter outlives the invented lifetime. In turn, this invented lifetime outlives the returned safe reference.

§ 2.1 Exclusivity

- `T%` is a *mutable safe reference*. It cannot alias other references to overlapping places.
- `const T%` is a *shared safe reference*. It may alias shared safe references to overlapping places, but may never overlap a mutable reference.

If lifetime safety can be guaranteed without lifetime parameters, why involve a new reference type `T%` at all? Why not perform this form of borrow checking on the existing lvalue- and rvalue-references `T&` and `T&&`? The answer is that safe references enforce *exclusivity* and legacy references do not. There may be one mutable reference to a place, or any number of shared (constant) references, but not both at the same time. This is the universal invariant of borrow checking. Borrow checking legacy reference types would break all existing code, because that code was written without upholding the exclusivity invariant.

Exclusivity is a program-wide invariant. It doesn't hinge on the safeness of a function.

- A safe function is sound for all valid inputs.
- An unsafe function has preconditions and may be unsound for some valid inputs.

“Valid” borrow and safe reference inputs don't mutably alias. This is something a function can just *assume*; it doesn't need to check and there's no way to check. Borrow checking upholds exclusivity even for unsafe functions (when compiled under the `[safety]` feature). There are other assumptions C++ programmers already make about the validity of inputs: for instance, references never hold null addresses. Non-valid inputs are implicated in undefined behavior.

With a desire to simplify, you may suggest “rather than adding a new safe reference type, just enforce exclusivity on lvalue- and rvalue-references when compiled under the `[safety]` feature.” But that makes the soundness problem worse. New code will *assume* legacy references don't mutably alias, but existing code doesn't uphold that invariant because it was written without considering exclusivity.

If safe code calls legacy code that returns a struct with a pair of references, do those references alias? Of course they may alias, but the parsimonious treatment claims that mutable references don't alias under the `[safety]` feature. We've already stumbled on a soundness bug.

Coming from the other direction, it may be necessary to form aliasing references just to use the APIs for existing code. Consider a call to `vec.push_back(vec[0])`. This is *impossible to express* without mutable aliasing: we form a mutable lvalue reference to `vec` and a `const` lvalue reference to one of `vec`'s elements. If safe code can't even form aliased lvalue references, it won't be able to use this API at all.

Exclusivity is a program-wide invariant on safe references. We need separate safe and unsafe reference types for both soundness and expressiveness.

[vector1.cxx – \(Compiler Explorer\)](#)

```
#include <vector>

void f1(std::vector<float>& vec, float& x) {
    // Do vec and x alias? If so, the push_back may invalidate x.
    vec.push_back(6);

    // Potential UB: x may have been invalidated by the push_back.
    x = 6;
}

int main() {
    std::vector<float> vec { 1.0f };

    // Legacy references permit aliasing.
    f1(vec, vec[0]);
}
```

This example demonstrates how perilous mutable aliasing in C++ is. In `f1`, the compiler doesn't know if `vec` and `x` alias. Pushing to the vector may cause a buffer resize and copy its data into a new allocation, invalidating existing references or pointers into the container. As C++ doesn't enforce exclusivity on legacy references, the code in `main` is legal, even though it leads to a use-after-free defect.

[vector2.cxx – \(Compiler Explorer\)](#)

```
#feature on safety
#include <cstdint>

template<typename T>
class Vec {
public:
    void push_back(T value) % safe;

    const T% operator[](size_t idx) const % safe;
    T% operator[](size_t idx) % safe;
};

void f2(Vec<float>% vec, float% x) safe {
    // Does push_back potentially invalidate x?
    // No! Exclusivity prevents vec and x from aliasing.
    vec.push_back(7);

    // Okay to store to x, because it doesn't point into vec's data.
    *x = 7;
}

int main() safe {
    Vec<float> vec { };
    mut vec.push_back(1);
}
```

```

    // Ill-formed: mutable borrow of vec between its mutable borrow and its
    // use
    f2(mut vec, mut vec[0]);
}

```

```

$ circle vector2.cxx
safety: during safety checking of int main() safe
  borrow checking: vector2.cxx:27:19
    f2(mut vec, mut vec[0]);
    ^
  mutable borrow of vec between its mutable borrow and its use
  loan created at vector2.cxx:27:10
    f2(mut vec, mut vec[0]);
    ^

```

Rewrite the example using our simplified safe references. In main, the user attempts to pass a safe reference to `vec` and a safe reference to one of its elements. This violates exclusivity, causing the program to be ill-formed.

Mutable safe references are prohibited from aliasing. Exclusivity is enforced by the same MIR analysis that polices Safe C++'s more general borrow type T^* . While enforcing exclusivity involves more complicated tooling, it simplifies reasoning about your functions. Since safe reference parameters don't alias, users don't even have to think about aliasing bugs. You're free to store to references without worrying about iterator invalidation or other side effects leading to use-after-free defects.

`exclusive1.cxx` – (Compiler Explorer)

```

#feature on safety

void f(int% x, int% y) safe;

void g(int& x, int& y) safe {
  unsafe {
    // Enter an unsafe block to dereference legacy references.
    // The precondition to the unsafe-block is that the legacy
    // references *do not alias* and *do not dangle*.
    f(%*x, %*y);
  }
}

void f(int% x, int% y) safe {
  // We can demote safe references to legacy references without
  // an unsafe block. There are no preconditions to enforce.
  g(&*x, &*y);
}

```

While safe references and legacy references are different types, they're inter-convertible. Converting a safe reference to legacy reference can be done safely, because it doesn't involve any preconditions. Function `f` converts a safe reference `x` to an lvalue reference with a dereference and reference-of: `&*x`. Going the other way is unsafe: the precondition of the *unsafe-block* is that the legacy references *do not alias* and *do not dangle*: `%*x`.

§ 2.2 Constraint rules

This proposal implements two sets of constraint rules. Free functions constrain return references by the shortest of the argument lifetimes. Non-static member functions constrain return references by the implicit object lifetime.

[lifetimes3.cxx](#) – (Compiler Explorer)

```
#feature on safety

const int% f1(const int% x, const int% y, bool pred) safe {
    // The return reference is constrained by all reference parameters: x
    // and y.
    return pred ? x : y;
}

struct Obj {
    const int% f2(const int% arg) const % safe {
        // Non-static member functions are constrained by the implicit
        // object lifetime.
        // It's OK to return `x`, because self outlives the return.
        return %x;
    }

    const int% f3(const int% arg) const % safe {
        // Error: arg does not outlive the return reference.
        return arg;
    }

    const int% f4(const self%, const int% arg) safe {
        // OK - f4 is a free function with an explicit self parameter.
        return arg;
    }

    int x;
};

int main() {
    int x = 1, y = 2;
    f1(x, y, true); // OK

    Obj obj { };
    obj.f2(x); // OK
    obj.f3(x); // Error
    obj.f4(x); // OK.
}

$ circle lifetimes3.cxx
safety: during safety checking of const int% Obj::f3(const int%) const %
safe
error: lifetimes3.cxx:18:12
    return arg;
        ^
function const int% Obj::f3(const int%) const % safe returns object with
lifetime SCC-ref-1, but SCC-ref-1 doesn't outlive SCC-ref-0
```

The definitions of free function `f1` and non-static member function `f2` compile, because they return function parameters that constrain the return type: the returned parameter *outlives* the returned reference. The non-static member function `f3` fails to compile, because the returned parameter *does not outlive* the the return type. In a non-static member function, only the implicit object parameter outlives the return type. `f4` returns a function parameter but compiles; it uses the explicit object syntax to gain the ergonomics of a non-static member function, but retains the constraint rules of a free function.

vector3.cxx – (Compiler Explorer)

#feature on safety

```
template<typename Key, typename Value>
class Map {
public:
    // Non-static member functions do not constrain the result object to
    // the function parameters.
    auto get1(const Key% key) % safe -> Value%;

    // Free function do constrain the result object to the function
    // parameters.
    auto get2(self%, const Key% key) safe -> Value%;
};

int main() safe {
    Map<float, long> map { };

    // Bind the key reference to a materialized temporary.
    // The temporary expires at the end of this statement.
    long% value1 = mut map.get1(3.14f);

    // We can still access value, because it's not constrained on the
    // key argument.
    *value1 = 1001;

    // The call to get2 constrains the returned reference to the lifetime
    // of the key temporary.
    long% value2 = mut map.get2(1.6186f);

    // This is ill-formed, because get2's key argument is out of scope.
    *value2 = 1002;
}
```

```
$ circle vector3.cxx
```

```
safety: during safety checking of int main() safe
```

```
  borrow checking: vector3.cxx:30:4
```

```
    *value2 = 1002;
```

```
    ^
```

```
  use of value2 depends on expired loan
```

```
  drop of temporary object float between its shared borrow and its use
```

```
  loan created at vector3.cxx:27:31
```

```
    long% value2 = mut map.get2(1.6186f);
```

```
    ^
```

The constraint rules for non-static member functions reflect the idea that resources are owned by class objects. Consider a map data structure that associates values with keys. The map may be specialized a key type that's expensive to copy, such as a string or another map. We don't want to compel the user to pass the key by value, because that may require copying this expensive type. Naturally, we pass by const reference.

However, the accessor only needs the key inside the body of the function. Once it locates the value, it should return a reference to that, unconstrained by the lifetime of the key argument. Consider passing a materialized temporary for a key: it goes out of scope at the end of the full expression. `get1` uses the non-static member function constraint rules. The caller can use the returned reference even after the key temporary goes out of scope. `get2` uses the free function constraint rules, which constrains the return type to all of its function parameters. This leaves the program ill-formed when the returned reference is used after the expiration of the key temporary.

In this model, lifetime constraints are not generally programmable, but that design still provides a degree of freedom in the form of non-static member functions.

[vector4.cxx](#) – (Compiler Explorer)

```
#feature on safety

template<typename Key, typename Value>
class Map {
public:
    // Lifetime elision rules constrain the return by self.
    auto get1(self^, const Key^ key) safe -> Value^;

    // Use explicit parameterizations for alternate constraints.
    auto get2/(a)(self^/a, const Key^/a key) safe -> Value^/a;
};

int main() safe {
    Map<float, long> map { };

    // Bind the key reference to a materialized temporary.
    // The temporary expires at the end of this statement.
    long^ value1 = mut map.get1(3.14f);

    // We can still access value, because it's not constrained on the
    // key argument.
    *value1 = 1001;

    // The call to get2 constrains the returned reference to the lifetime
    // of the key temporary.
    long^ value2 = mut map.get2(1.6186f);

    // This is ill-formed, because get2's key argument is out of scope.
    *value2 = 1002;
}
```



```

$ circle vector4.cxx
safety: during safety checking of int main() safe
  borrow checking: vector4.cxx:29:4
    *value2 = 1002;
    ^
  use of value2 depends on expired loan
  drop of temporary object float between its shared borrow and its use
  loan created at vector4.cxx:26:31
    long^ value2 = mut map.get2(1.6186f);
    ^

```

The general borrow type $T^\wedge$ has programmable constraints. The map above declares accessor functions. `get1` relies on lifetime elision to constrain the result object by the `self` parameter. This is equivalent to the non-static member function constraint rule. We can call `get1` and use the returned reference even after the key temporary goes out of scope.

`get2` includes lifetime annotations to constrain the returned reference by both the `self` and key parameters. This is like the free function constraint rules. The program fails borrow checking when the returned reference `value2` is used after the expiration of its key temporary.

§ 3 Second-class references

References can be taxonimized into two classes: [second-class](#)

- First-class references can pass data into functions, be returned from functions, made into objects and be stored in structures.
- Second-class references can pass data into functions but cannot be returned from functions, made into objects or stored in structures.

Parameter-passing directives like `in` and `inout` are equivalent to second-class references. The *mutable value semantics* [mutable-value-semantics](#) model uses parameter-passing directives to pass objects to functions by reference without involving the complexity of a borrow checker.

```
void func(Vec<float>% vec, float% x) safe;
```

In this fragment, the reference parameters `vec` and `x` serve as *second-class references*. The compiler can achieve memory safety without involving the complexity of borrow checking. Both references point at data that outlives the duration of the call to `func`. Exclusivity is enforced at the point of the call, which prevents `vec` and `x` from aliasing. Since `vec` and `x` don't alias, resizing or clearing `vec` cannot invalidate the `x` reference.

The safe references presented here are more powerful than second-class references. While they don't support all the capabilities of borrows, they can be returned from functions and made into objects. The compiler must implement borrow checking to support this additional capability.

Borrow checking operates on a function lowering called mid-level IR (MIR). A fresh region variable is provisioned for each local variable with a safe reference type. Dataflow analysis populates each region variable with the liveness of its reference. Assignments and function calls involving references generate *lifetime constraints*. The compiler *solves the constraint equation* to find the liveness of each *loan*. All instructions in the MIR are scanned for *conflicting actions* with any of the loans in scope at that point. Examples of conflicting actions are stores to places with live shared borrows or loads from places with live mutable borrows. Conflicting actions raise borrow checker errors.

The Hylo[hylo] model is largely equivalent to this model and it requires borrow checking technology. `let` and `inout` parameter directives use mutable value semantics to ensure memory safety for objects passed by reference into functions. But Hylo also supports returning references in the form of subscripts:

Array.hylo

```
public conformance Array: Collection {
    ...
    public subscript(_ position: Int): Element {
        let {
            precondition((position >= 0) && (position < count()), "position is
                out of bounds")
            yield pointer_to_element(at: position).unsafe[]
        }
        inout {
            precondition((position >= 0) && (position < count()), "position is
                out of bounds")
            yield &(pointer_to_element(at: position).unsafe[])
        }
    }
}
```

Subscripts are reference-returning *coroutines*. Coroutines with a single yield point are split into two normal functions: a ramp function that starts at the top and returns the expression of the yield statement, and a continuation function which resumes after the yield and runs to the end. Local state that's live over the yield point must live in a *coroutine frame* so that it's available to the continuation function. These Array subscripts don't have instructions after the yield, so the continuation function is empty and hopefully optimized away.

```
template<typename T>
struct Vec {
    const T% operator[](size_t idx) const % safe;
    T% operator[](size_t idx) % safe;
};
```

The Hylo Array subscripts are lowered to reference-returning ramp functions exactly like their C++ Vec counterparts. For both languages, the borrow checker relates lifetimes through the function arguments and out the result object. This isn't the simple safety of second-class references/mutable value semantics. This is full-fat live analysis.

Safe references without lifetime annotations shields users from dealing with a new degree of freedom, but it doesn't simplify the static analysis that upholds lifetime safety. To prevent use-after-free defects, compilers must still lower functions to mid-level IR, compute non-lexical lifetimes[nll] and solve the constraint equation. When it comes to returning references, in for a penny, in for a pound.

Since Circle has already made the investment in borrow checking, adding simplified safe references was an easy extension. If the community is able to fill in our gaps in knowledge around this sort of reference, the compiler could accommodate those advances as well.

§ 4 Other aspects of safety

As detailed in the Safe C++[safecpp] proposal, there are four categories of memory safety:

1. **Lifetime safety** - This proposal advances a simpler form of safe references that provides safety against use-after-free defects. The feature is complementary with borrow types $T^{\wedge}$ that take lifetime arguments. Both types can be used in the same translation unit, and even the same function, without conflict.
2. **Type safety** - Relocation must replace move semantics to eliminate unsafe null pointer exposure. Choice types and pattern matching must be included for safe modeling of optional types.
3. **Thread safety** - The send and sync interfaces account for which types can be copied and shared between threads.
4. **Runtime checks** - The compiler automatically emits runtime bounds checks on array and slice subscripts. It emits checks for integer divide-by-zero and `INT_MIN / -1`, which are undefined behavior. Conforming safe library functions must panic to prevent out-of-bounds access to heap allocations.

Most critically, the *safe-specifier* is added to a function's type. Inside a safe function, only safe operations may be used, unless escaped by an *unsafe-block*.

C++ must adopt a new standard library with a safe API, which observes all four categories of safety. We need new tooling. *But it's not the case that we have to rewrite all C++ code.* Time has already shaken out most of the vulnerabilities in old code. As demonstrated by the recent Android study on memory safety[android], the benefits of rewriting are often not worth the costs. What we have to prioritize is the transition to safe coding practices[safe-coding] for new code.

§ 5 Achieving first-class references

The presented design is as far as I could go to address the goal of “memory safety without lifetime parameters.” But safe references aren't yet powerful enough to replace all the unsafe mechanisms necessary for productivity in C++. We need support for safe versions of idioms that are central to the C++ experience, such as:

- Iterators.
- Views like `string_view` and `span`.
- RAII types with reference semantics.

Let's consider RAII types with reference semantics. An example is `std::lock_guard`, which keeps a reference to a mutex. When the `lock_guard` goes out of scope its destructor calls `unlock` on the mutex. This is a challenge for safe references, because safe reference data members aren't supported. Normally those would require lifetime parameters on the containing class.

Robust support for user-defined types with reference data members isn't just a convenience in a safe C++ system. It's a necessary part of *interior mutability*, the core design pattern for implementing shared ownership of mutable state (think safe versions of `shared_ptr`).

What are some options for RAII reference semantics?

- **Coroutines.** This is the Hylo strategy. The `ramp` function locks a mutex and returns a safe reference to the data within. The continuation unlocks the mutex. The reference to the mutex is kept in the coroutine frame. But this still reduces to supporting structs with reference data members. In this case it's not a user-defined type, but a compiler-defined coroutine frame. The coroutine solution is an unidiomatic fit for C++ for several reasons: static allocation of the coroutine frame requires exposing the definition of the coroutine to the caller, which breaks C++'s approach to modularity; the continuation is called immediately after the last use of the yielded reference, which runs counter to expectation that cleanup runs at the end of the enclosing scope; and since the continuation is called implicitly, there's nothing textual on the caller side to indicate an unlock.
- **Defer expressions.** Some garbage-collected languages include *defer* expressions, which run after some condition is met. We could defer a call to the mutex unlock until the end of the enclosing lexical scope. This has the benefit of being explicit to the caller and not requiring computation of a coroutine frame. But it introduces a fundamental new control flow mechanism to the language with applicability that almost perfectly overlaps with destructors.
- **Destructors.** This is the idiomatic C++ choice. A local object is destroyed when it goes out of scope (or is dropped, with the Safe C++ `drop` keyword). The destructor calls the mutex unlock.

It makes sense to strengthen safe references to support current RAII practice. How do we support safe references as data members? A reasonable starting point is to declare a class as having *safe reference semantics*. `class name %;` is a possible syntax. Inside these classes, you can declare data members and base classes with safe reference semantics: that includes both safe references and other classes with safe reference semantics.

```
class lock_guard % {  
    // Permitted because the containing class has safe reference semantics.
```

```

std2::mutex% mutex;
public:
    ~lock_guard() safe {
        mutex.unlock();
    }
};

```

The constraint rules can apply to the new `lock_guard` class exactly as it applies to safe references. Returning a `lock_guard` constrains its lifetime by the lifetimes of the function arguments. Transitively, the lifetimes of the data members are constrained by the lifetime of the containing class.

Unfortunately, we run into problems immediately upon declaring member functions that take safe reference objects or safe reference parameter types.

```

class string_view %;

template<typename T>
class info % {
    // Has reference semantics, but that's okay because the containing class
    // does.
    string_view sv;
public:
    void swap(info% rhs) % safe;
};

```

Consider an `info` class that has *safe reference semantics* and keeps a `string_view` as a data member. The `string_view` also has reference semantics, so it constrains the underlying string that owns the data. Declare a non-static member function that binds the implicit object with the `% ref-qualifier` and also takes an `info` by safe reference. This is uncharted water. The implicit object type `info` has reference semantics, yet we're taking a reference to that with `swap` call. We're also taking a reference to `info` in the function parameter. How do we deal with references to references? The existing constraint rules only invent a single lifetime: if we used those, we'd be clobbering the lifetime of the inner `string_view` member.

There's a big weakness with the safe reference type `T%`: it's under-specified when dealing with references to references. We need a fix that respects the lifetimes on the class's data members.

[lifetimes5.cxx](#) – (Compiler Explorer)

```

#feature on safety

class string_view/(a) {
    // Keep a borrow to a slice over the string data.
    const [char; dyn]^/a p_;
public:
};

class info/(a) {
    // The handle has lifetime /a.

```

```

    string_view/a sv;

public:
    void swap(self^, info^ rhs) safe {
        string_view temp = self->sv;
        self->sv = rhs->sv;
        rhs->sv = temp;
    }
};

void func/(a)(info/a^ lhs, info/a^ rhs) safe {
    lhs.swap(rhs);
}

void func2(info^ lhs, info^ rhs) safe {
    lhs.swap(rhs);
}

```

Rust and Safe C++ have a way to keep the lifetime of the `string_view` member distinct from the lifetimes of the `self` and `rhs` references: lifetime parameters. `func` assumes that the `string_views` of its parameters come from sources with overlapping lifetimes, so it declares a lifetime parameter `/a` that's common to both parameters. The lifetimes on the two references are created implicitly by elision, as they don't have to be related in the `swap` call. `func` compiles and doesn't clobber the lifetimes of the contained `string_views`.

```

safety: during safety checking of void func2(info^, info^) safe
error: lifetimes5.cxx:26:12
    lhs.swap(rhs);
        ^
    function void func2(info^, info^) safe returns object with lifetime #0,
        but #0 doesn't outlive #2

error: lifetimes5.cxx:26:3
    lhs.swap(rhs);
    ^
    function void func2(info^, info^) safe returns object with lifetime #2,
        but #2 doesn't outlive #0

```

Compiling `func2` raises borrow checker errors. Instead of providing explicit lifetime annotations that relate the lifetimes of the `lhs` and `rhs` `info` types, lifetime elision create four distinct lifetimes: `#0` for the `lhs info`, `#1` for the `lhs info^`, `#2` for the `rhs info` and `#3` for the `rhs info^`. The `lhs.swap(rhs)` call relates the lifetimes of the operands through the common lifetime `/a`. But these lifetimes aren't related! The compiler has no information whether `#0` outlives `#2` or vice versa. Since the lifetimes aren't related in `func2`'s declaration, the program is rejected as ill-formed.

This contrasts with the safe reference constraint rules, which would assign the same lifetime to all four lifetime binders and clobber the `string_view` lifetimes, causing a borrow checker failure further from the source and leaving the developer without the possibility of a fix.

§ 6 Lifetime parameters

If there's a community-wide research effort among compiler experts to evolve safe references it may be possible to get them into a state to support the abstractions most important for C++. But soundness reasoning is very subtle work. As you increase the indirection capability of safe references, you invite networks of dependencies of implied constraints and variances. This increases complexity for the compiler implementation and puts a mental burden on the authors of unsafe code to properly uphold the invariants assumed by safe references. A research project must produce *soundness doctrine*, which is essential guidance on how to interface safe and unsafe systems while upholding the soundness invariants of the program.

But we don't have to do the research. There's already a solution that's been deployed in a successful production toolchain for a decade: *lifetime parameters* as used in Rust. The soundness doctrine for writing unsafe code that upholds the invariants established by lifetime parameters is described in the Rustnomicon[\[rustnomicon\]](#).

This is the only known viable solution for first-class safe references without garbage collection. It's a critical lifeline that addresses an existential problem facing C++. By adopting lifetime parameters, C++ can achieve safety parity with the security community's favored languages.

Consider common objections to Rust's lifetime-annotation flavor of borrow checking:

1. **You need heavy annotations.** This concern is misplaced. Are you intrigued by mutable value semantics, parameter-passing directives or second-class references? Borrow checking gives you those, without ever having to write lifetime arguments. If your function only uses references as parameters, elision implicitly annotates them in a way that can't fail. You only have to involve lifetime arguments when going beyond the capabilities of second-class references or mutable value semantics. More advanced usages such as the implementation of iterators, views and RAI wrappers with reference semantics are where annotations most often appear, because those designs deal with multiple levels of references.
2. **Borrow checking doesn't permit patterns such as self-references.** It's true that checked references are less flexible than unsafe references or pointers, but this objection is at odds with the claim that lifetime parameters are too burdensome. Lifetime parameters *increase* the expressiveness of safe references. Additionally, they can reference things important to C++ users that a garbage collection can't, such as variables on the stack. Do we want more expressive references at the cost of annotations, or do we want to get rid of lifetime parameters to make a simpler language? Those are opposing goals.
3. **Borrow checking with lifetimes is too different from normal C++.** Borrow checking is the safety technology most similar to current C++ practice. This model replaces unchecked references with checked references. Other safety models get rid of reference types entirely or replace them with garbage collection which is

incompatible with C++'s manual memory management and RAIL. The design philosophy of borrow checking is to take normal references but constrain them to uses that can be checked for soundness by the compiler.

It's not surprising that the C++ community hasn't discovered a better way to approach safe references than the lifetime parameter model. After all, there isn't a well-funded effort to advance C++ language-level lifetime safety. But there is in the Rust community. Rust has made valuable improvements to its lifetime safety design. Lots of effort goes into making borrow checking more permissive: The integration of mid-level IR and non-lexical lifetimes in 2016 revitalized the toolchain. Polonius[polonius] approaches dataflow analysis from the opposite direction, hoping to shake loose more improvements. Ideas like view types[view-types] and the sentinel pattern[sentinel-pattern] are being investigated. But all this activity has not discovered a mechanism that's superior to lifetime parameters for specifying constraints. If something had been discovered, it would be integrated into the Rust language and I'd be proposing to adopt *that* into C++. For now, lifetime parameters are the best solution that the world has to offer.

The US government and major players in tech including Google[secure-by-design] and Microsoft[ms-vulnerabilities] are telling industry to transition to memory-safe languages because C++ is too unsafe to use. There's already a proven safety technology compatible with C++'s goals of performance and manual memory management. If the C++ community rejects this robust safety solution on the grounds of slightly inconvenient lifetime annotations, and allows C++ to limp forward as a memory-unsafe language, can it still claim to care about software quality? If the lifetime model is good enough for a Rust, a safe language that is enjoying snowballing investment in industry, why is it not good enough for C++?

Finally, adoption of this feature brings a major benefit even if you personally want to get off C++: It's critical for **improving C++/Rust interop**. Your C++ project is generating revenue and there's scant economic incentive to rewrite it. But there is an incentive to pivot to a memory-safe language for new development, because new code is how vulnerabilities get introduced.[android] Bringing C++ closer to Rust with the inclusion of *safe-specifier*, relocation, choice types, and, importantly, lifetime parameters, reduces the friction of interfacing the two languages. The easier it is to interoperate with Rust, the more options and freedom companies have to fulfill with their security mandate.[rust-interop]

§ 7 References

[android] Eliminating Memory Safety Vulnerabilities at the Source.

<https://security.googleblog.com/2024/09/eliminating-memory-safety-vulnerabilities-Android.html?m=1>

[hylo] Borrow checking Hylo.

<https://2023.splashcon.org/details/iwaco-2023-papers/5/Borrow-checking-Hylo>

[ms-vulnerabilities] We need a safer systems programming language.
<https://msrc.microsoft.com/blog/2019/07/we-need-a-safer-systems-programming-language/>

[mutable-value-semantics] Implementation Strategies for Mutable Value Semantics.
https://www.jot.fm/issues/issue_2022_02/article2.pdf

[nll] The Rust RFC Book - Non-lexical lifetimes.
<https://rust-lang.github.io/rfcs/2094-nll.html>

[polonius] Polonius revisited.
<https://smallcultfollowing.com/babysteps/blog/2023/09/22/polonius-part-1/>

[rust-interop] Improving Interoperability Between Rust and C++.
<https://security.googleblog.com/2024/02/improving-interoperability-between-rust-and-c.html>

[rustnomicon] Rustnomicon – The Dark Arts of Unsafe Rust.
<https://doc.rust-lang.org/nomicon/intro.html>

[safe-coding] Tackling cybersecurity vulnerabilities through Secure by Design.
<https://blog.google/technology/safety-security/tackling-cybersecurity-vulnerabilities-through-secure-by-design/>

[safecpp] P3390 – Safe C++.
<https://safecpp.org/draft.html>

[second-class] Second-Class References.
<https://borretti.me/article/second-class-references>

[secure-by-design] Secure by Design : Google’s Perspective on Memory Safety.
<https://research.google/pubs/secure-by-design-googles-perspective-on-memory-safety/>

[sentinel-pattern] After NLL: Moving from borrowed data and the sentinel pattern.
<https://smallcultfollowing.com/babysteps/blog/2018/11/10/after-nll-moving-from-borrowed-data-and-the-sentinel-pattern/>

[view-types] View types for Rust.
<https://smallcultfollowing.com/babysteps/blog/2021/11/05/view-types/>